Technical University Munich

School of Engineering and Design

Report for Deep Learning for PDEs in Engineering Physics

Álvaro Román Sánchez

03818482

Supervisors: Dr. Yaohua Zang & Vincent Scholz

July 8, 2026

Contents

Chapter 1

Problem A

1.1 Problem Description

Problem A consists in recovering the Young's modulus $k(x)$ of a 1D rod made of a linearly elastic material given noisy measurements of its displacement field $u(x)$. The Young's modulus is not homogeneous along the length of the rod. The PDE governing this system is

$$-\frac{d}{dx}\left(k(x)\frac{du}{dx}\right) = f, \quad x \in (0, L) \tag{1.1}$$

where $f = 9.81$ is the axial load applied to the rod and $L = 1$ is the rod's length.

The rod's displacement is fixed at both ends, and therefore the PDE is subject to the folling initial conditions:

$$u(0) = u(L) = 0 \tag{1.2}$$

This problem can be categorized as an inverse problem, since we are given observations of the solution $u(x)$ of the PDE and are asked to find the material field $k(x)$, instead of the other way around (which would be deemed a forward problem).

1.2 Methodology

1.2.1 The method selection

The method chosen for this problem is to use a Variational Physics-Informed Neural Network (VPINN). This type of neural network is chosen due to its applicability to inverse problems, as opposed other deep learning methods like Deep Ritz, in which inverse problems are not directly possible. Since we are solving for a single material field, it is not necessary to learn an operator. Thus, Deep Neural Operators (DNOs) do not fit this type of problem.

A simple visual inspection of the noisy $u(x)$ data reveals it has a simple geometry, which VPINNs are known to perform well on due to their global polynomial basis. Furthermore, the problem was also attempted using a Weak Adversarial Network (WAN) approach and a Particle Weak Neural Network (ParticleWNN) approach, but the VPINN approach was found to give the best performance on the problem. Additionally, the problem is 1-dimensional, making VPINN a competitive option for this task, as well as making it simple to implement. The reasons listed above make VPINN an excellent method in comparison to other physics-informed methods for this specific problem.

1.2.2 The loss function

The loss function used for this task is composed of 3 terms: A boundary loss, an observation loss and PDE loss. The boundary loss enforces that the boundary conditions $g(x)$ are satisfied by $u_\theta(x)$, and is defined as

$$\mathcal{L}_{\text{bd}} = \frac{1}{N_{\text{bd}}} \sum_{i=1}^{N_{\text{bd}}} |u_\theta(x_i) - g(x_i)|^2 \tag{1.3}$$

$$= \frac{1}{2} |u_\theta(0) - 0|^2 + \frac{1}{2} |u_\theta(L) - 0|^2 \tag{1.4}$$

where $N_{\text{bd}} = 2$ since there are 2 boundary points in the domain.

The observation loss ensures that the network's prediction $u_\theta(x)$ stays consistent with the noisy measurements u^{obs} at the sensor locations x^{obs}. It is defined as

$$\mathcal{L}_{\text{obs}} = \frac{1}{N_{\text{obs}}} \sum_{i=1}^{N_{\text{obs}}} \left| u_\theta(x_i^{\text{obs}}) - u_i^{\text{obs}} \right|^2 \tag{1.5}$$

where $N_{\text{obs}} = 500$ since there are 500 noisy observations.

Finally, the PDE loss ensures that the pair of predicted functions ($u_\theta(x)$ and $k_\theta(x)$) satisfy the governing PDE. It is computed as the sum of squared weak residuals R_n of each test function $v_n(x)$ for $n = 1, \ldots, N_{\text{test}}$, where $v_n(x)$ is the n^{th} Legendre polynomial. The residuals are given by the weak form of the PDE, which is approximated numerically by Gauss-Legendre quadrature.

$$R_n = \int_0^L (k_\theta(x) \nabla u_\theta(x) \nabla v_n(x) - f(x) v_n(x))\, dx \tag{1.6}$$

$$\approx \sum_{j=1}^{N_{\text{int}}} (k_\theta(x_j) \nabla u_\theta(x_j) \nabla v_n(x_j) - f(x_j) v_n(x_j))\, w_j \tag{1.7}$$

where N_{int} is the number of integral points to use in the quadrature, chosen here to be $N_{\text{int}} = 100$, and w_j are the quadrature weights.

The PDE loss is then given by

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} R_i^2 \tag{1.8}$$

The number of test functions N_{test} is chosen here to be $N_{\text{test}} = 100$.

The total training loss $\mathcal{L}_\theta$ is computed as a weighted sum of the 3 losses mentioned previously, where the weights associated to each loss are hyperparameters.

$$\mathcal{L}_\theta = w_{\text{bd}} \mathcal{L}_{\text{bd}} + w_{\text{obs}} \mathcal{L}_{\text{obs}} + w_{\text{PDE}} \mathcal{L}_{\text{PDE}} \tag{1.9}$$

1.2.3 The network structure

The network structure used consists of 2 Multilayer Perceptrons (MLPs), one for the estimation of the Young's modulus $k(x)$ and the other for the estimation of the displacement $u(x)$. Both networks are trained simultaneously during the training process. The network architectures of said MLPs can be found in Table 1.1 and Table 1.2.

Table 1.1: Architecture of MLP for $k(x)$

Layer	Size / Neurons	Activation Function	Notes
Input Layer	1	$\sigma_A(x)$	-
Hidden Layer 1	200	$\sigma_A(x)$	Fully connected
Hidden Layer 2	60	$\sigma_A(x)$	Fully connected
Hidden Layer 3	60	$\sigma_A(x)$	Fully connected
Hidden Layer 4	200	$\sigma_A(x)$	Fully connected
Output Layer	1	Softplus	For $k_\theta(x) > 0$

The custom activation function $\sigma_A(x)$ used for all layers except the output layer is

$$\sigma_A(x) = \sin(\pi x + \pi) + \sin(x) \tag{1.10}$$

This type of sinusoidal activation is common in a type of networks called Sinusoidal Representation Networks (SIRENs), which use sinusoidal functions to represent signals with fine details [1]. One of the main advantages of this function is that it is infinitely differentiable at every location, which enables back-propagation. Other common activations like Sigmoid, Tanh and ReLU we tried for the hidden layers, but all yielded worse performance.

The activation for the output layer is chosen to be Softplus rather than ReLU to enforce the predicted Young's modulus $k_\theta(x)$ to be positive while still ensuring differentiability in the whole domain.

Table 1.2: Architecture of MLP for $u(x)$

Layer	Size / Neurons	Activation Function	Notes
Input Layer	1	$\sigma_A(x)$	-
Hidden Layer 1	200	$\sigma_A(x)$	Fully connected
Hidden Layer 2	60	$\sigma_A(x)$	Fully connected
Hidden Layer 3	60	$\sigma_A(x)$	Fully connected
Hidden Layer 4	200	$\sigma_A(x)$	Fully connected
Output Layer	1	Linear	-

The architecture of the MLP for the prediction $u_\theta(x)$ is almost identical to the other MLP, except for the use of a Linear activation on the output layer. This ensures that $u_\theta(x)$ can take values both greater and smaller than 0.

1.2.4 The code link

Please provide the hyperlink (e.g., GitHub, Google Colab, or Kaggle Notebook) to your original code for solving this problem.

1.3 Numerical Experiments

1.3.1 Experiment Setups

The hyperparameters used in the experimental setup are listed in Table 1.3. Since both MLPs were trained simultaneously, all hyperparameters were kept the same for both networks in order to simplify the process. To achieve this simultaneous parameter updating, the full lists of parameters from both MLPs are passed into the optimizer, as such:

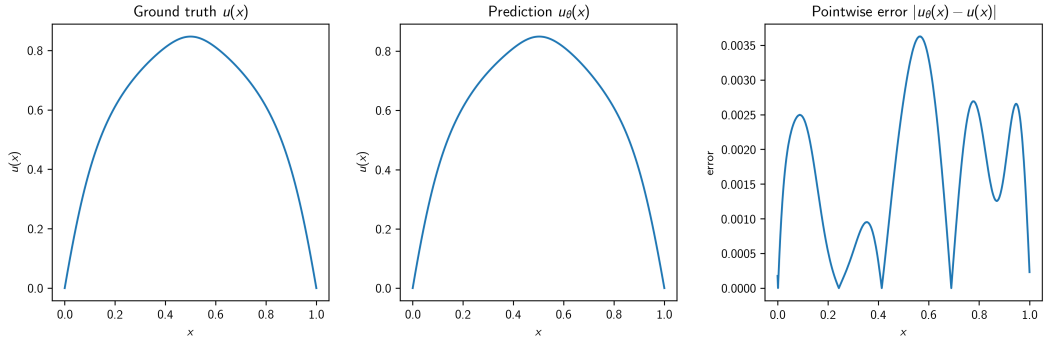
```
optimizer = torch.optim.Adam(params=list(model_u.parameters())+list(
  model_k.parameters()), lr=0.001)
```

Table 1.3: Numerical setup for training both VPINNs in PyTorch

Parameter	Value
w_{bd}	1
w_{obs}	5
w_{PDE}	1
Optimizer	Adam
Learning Rate (initial)	1×10^{-3}
Batch Size	500
Number of Epochs	2500
Learning Rate Scheduler	StepLR
Decay Rate (α/N epochs)	0.667/250
Hardware	NVIDIA RTX 3050 Laptop (4GB)
Precision	Float32
Early Stopping	No

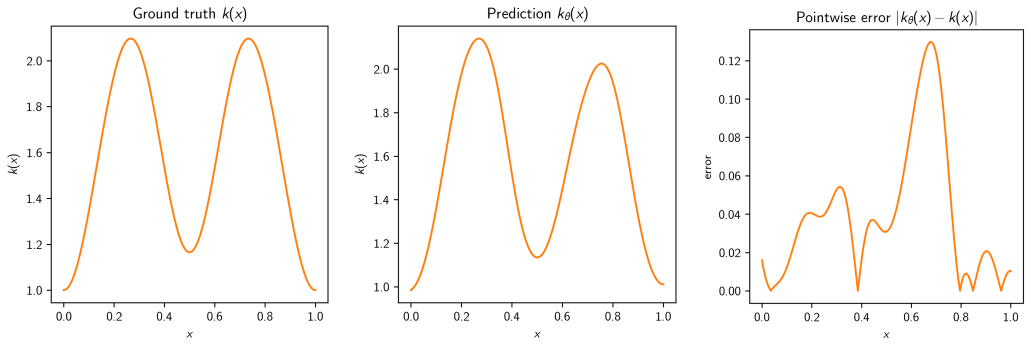
1.3.2 Numerical Results

The results of the experiment are shown in Figure 1.1 and Figure 1.2. The loss history for both MLPs is shown in Figure 1.3.



(a) Ground truth $u(x)$ (b) Prediction $u_\theta(x)$ (c) Pointwise error $|u(x) - u_\theta(x)|$

Figure 1.1: Experiment results of MLP for displacement $u(x)$



(a) Ground truth $k(x)$ (b) Prediction $k_\theta(x)$ (c) Pointwise error $|k(x) - k_\theta(x)|$

Figure 1.2: Experiment results of MLP for Young's modulus $k(x)$

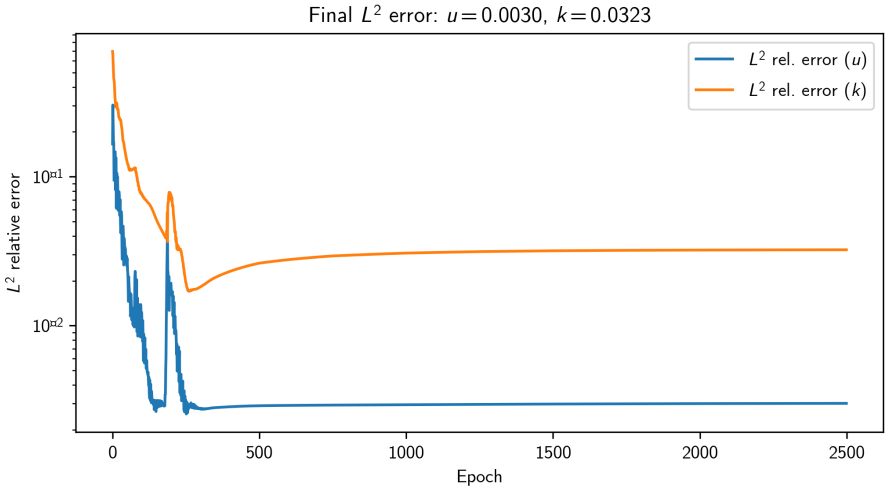


Figure 1.3: Evolution of L^2 relative error for Problem A

1.3.3 Discussion

As can be seen in Figure 1.1 and Figure 1.2, the VPINN performs remarkably well, especially on the displacement field $u(x)$. The Young's modulus $k(x)$ is approximated well in terms of the general shape of the function, although some pointwise errors still remain. This was to be expected, since after all the dataset contained labeled (noisy) measurements of $u(x)$, but none of $k(x)$, and it is known that physics-informed models can greatly benefit from labeled data even in small amounts.

As shown in Figure 1.3, the L^2 relative error has clearly been minimized to its full potential, indicating that further training with the same hyperparameters and model architecture would not benefit the performance any further. The final error for $k(x)$ is greater than that of $u(x)$, which makes sense when we visually analyze both their predictions as mentioned before.

Chapter 2

Problem B

2.1 Problem Description

Problem B consists in solving for the pressure field $u(x)$ in a 2-dimensional heterogeneous porous medium. The medium is composed of two distinct material phases with significantly different permeabilities, given by $\mu(x)$ which is provided in the dataset.

$$\mu(x) = \begin{cases} \mu_1 = 10, & x \in \Omega_1 \quad \text{(high permeability phase)} \\ \mu_2 = 2, & x \in \Omega_2 \quad \text{(low permeability phase)} \end{cases} \tag{2.1}$$

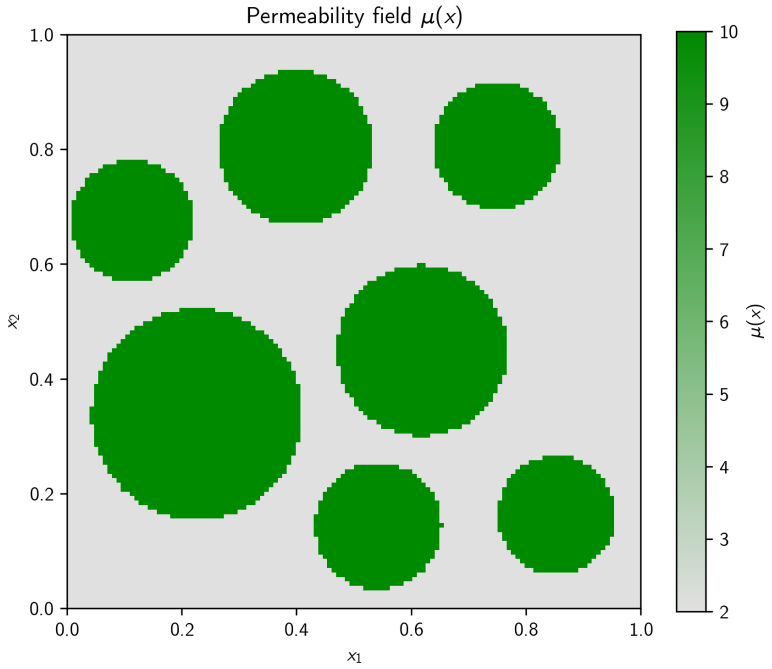


Figure 2.1: Permeability field $\mu(x)$

The governing PDE for this problem is the Darcy flow equation, namely

$$-\nabla \cdot (\mu(x)\nabla u(x)) = f, \quad x \in \Omega = [0,1]^2 \tag{2.2}$$

where $f = 0$ is the source term (no source or sink in this case) and Ω is the computational domain containing the sub-domains with different permeabilities Ω_1, Ω_2.

The PDE is subject to the following boundary conditions $g_D(x)$:

$$g_D(x) = \begin{cases} 1, & x_1 = 0 \\ 0, & x_1 = 1 \\ \text{linear interpolation}, & x_2 = 0 \text{ or } 1 \end{cases} \tag{2.3}$$

2.2 Methodology

2.2.1 The method selection

The method of choice for this problem is to use a ParticleWNN approach. The reason for this choice is that this method applicable to forward problems (like Problem B) and performs well on low dimensional data. Since we are learning the solution for a single material field, a DNO is not necessary for this problem. ParticleWNN chosen over other weak-form methods due to its ability to handle sharp gradients and locally complex solutions. This is suitable for the specific dataset at hand, given that the material field $\mu(x)$ has instantaneous changes in permeability rather than smooth transitions, as can be seen in Figure 2.1.

2.2.2 The loss function

The loss function used in this problem comprises 2 terms: a boundary condition loss and a PDE loss. The boundary condition loss is defined as the Mean Squared Error (MSE) between the predicted pressure $u_\theta(x)$ at the boundary and the true boundary pressure $g_D(x)$:

$$\mathcal{L}_{\text{bd}} = \frac{1}{|\partial\Omega|} \sum_{x \in \partial\Omega} |u_\theta(x) - g_D(x)|^2 \tag{2.4}$$

where $\partial\Omega$ is the boundary of the computational domain.

To compute the PDE loss, we first compute the weak residuals R_n of each test function $v_n(x)$. The test functions used in this case are Wendland Compactly Supported Radial Basis Functions (CSRBFs). The centers of said CSRBFs are found by fisrt sampling $N_c = 10^4$ "particles" uniformly inside the computational domain, then every training epoch taking a random subset of them for the computation of the weak residuals. The residuals are approximated by Gauss-Legendre quadrature, as follows:

$$R_n = \int_\Omega (\mu(x)\nabla u_\theta(x)\nabla v_n(x) - f(x)v_n(x))\, dx \tag{2.5}$$

$$\approx \sum_{j=1}^{N_{\text{int}}} (\mu(x_j)\nabla u_\theta(x_j)\nabla v_n(x_j) - f(x_j)v_n(x_j))\, w_j \tag{2.6}$$

where N_{int} is the number of integral points to use in the quadrature, chosen here to be $N_{\text{int}} = 15$, and w_j are the quadrature weights. The permeability field $\mu(x)$ is only defined at discrete grid locations (provided by the dataset), however it must be evaluated at quadrature points for the calculation of the R_n. To do this, bilinear interpolation between the grid points is used.

The PDE loss is then given by

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{test}}} \sum_{i=1}^{N_{\text{test}}} R_i^2 \tag{2.7}$$

The number of test functions N_{test} is chosen here to be $N_{\text{test}} = 200$.

The total training loss $\mathcal{L}_\theta$ is computed as a weighted sum of the 2 losses mentioned previously, where the weights associated to each loss are hyperparameters.

$$\mathcal{L}_\theta = w_{\text{bd}}\mathcal{L}_{\text{bd}} + w_{\text{PDE}}\mathcal{L}_{\text{PDE}} \tag{2.8}$$

2.2.3 The network structure

The network structure used for Problem B consists of one MLP with fully connected layers. The network architecture of said MLP can be found in Table 2.1.

Table 2.1: Network architecture of ParticleWNN for Problem B

Layer	Size / Neurons	Activation Function	Notes
Input Layer	2	$\sigma_B(x)$	-
Hidden Layer 1	200	$\sigma_B(x)$	Fully connected
Hidden Layer 2	50	$\sigma_B(x)$	Fully connected
Hidden Layer 3	50	$\sigma_B(x)$	Fully connected
Hidden Layer 4	100	$\sigma_B(x)$	Fully connected
Output Layer	1	Linear	-

The custom activation function $\sigma_B(x)$ used for all layers except the output layer is

$$\sigma_B(x) = \sin(\pi x + \pi) + \sin(x) \tag{2.9}$$

which identical to the activation used for the hidden layers in Problem A, as seen in equation (1.10). Once again the same reasoning is applied: the sinusoidal activation is used to try to capture fine details [1]. Other common activations including Sigmoid, Tanh or ReLU were also tried in this problem, but this custom activation yielded the best performance.

The Linear activation of the output layer ensures that $u_\theta(x)$ can be both greater and smaller than 0.

2.2.4 The code link

Please provide the hyperlink (e.g., GitHub, Google Colab, or Kaggle Notebook) to your original code for solving this problem.

2.3 Numerical Results

2.3.1 Experiment Setups

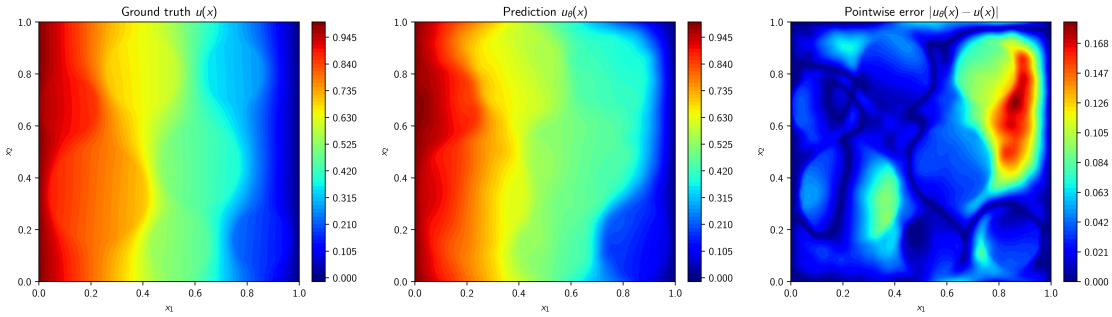
The hyperparameters used in the experimental setup are listed in Table 2.2.

Table 2.2: Numerical setup for training the ParticleWNN in PyTorch

Parameter	**Value**
w_{bd}	1×10^6
w_{PDE}	1
Optimizer	Adam
Learning Rate (initial)	1×10^{-2}
Batch Size	200
Number of Epochs	60
Learning Rate Scheduler	StepLR
Decay Rate (α/N epochs)	0.9/3
Hardware	NVIDIA RTX 3050 Laptop (4GB)
Precision	Float32
Early Stopping	No

2.3.2 Numerical Results

The results of the experiment are shown in Figure 2.2. The loss history is shown in Figure 2.3.



(a) Ground truth $u(x)$ (b) Prediction $u_\theta(x)$ (c) Pointwise error $|u(x) - u_\theta(x)|$

Figure 2.2: Experiment results of Problem B

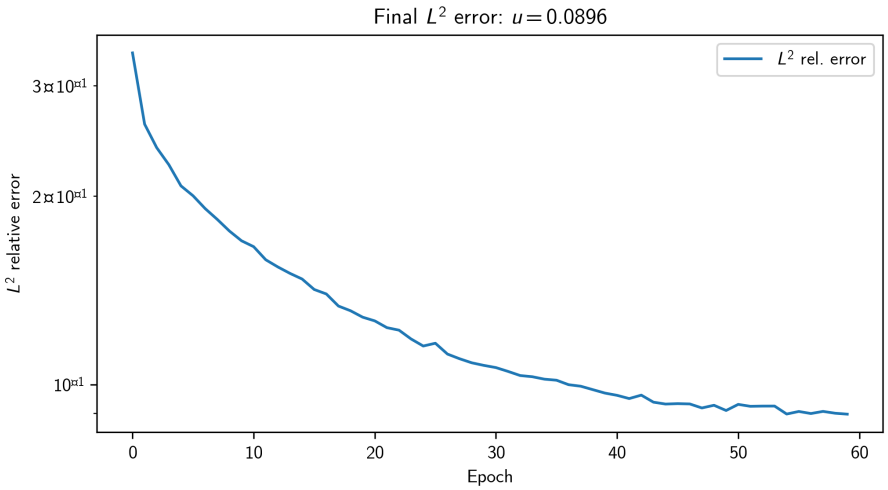


Figure 2.3: Evolution of L^2 relative error for Problem B

2.3.3 Discussion

As can be seen in Figure 2.2, the ParticleWNN is able to capture the general shape of the pressure field, but it struggles to capture the finer details. This could possibly be due to the number of parameters in the network being too small, or simply an incorrect choice of hyperparameters. Nevertheless, the boundary conditions are well respected.

Looking at the L^2 relative error in Figure 2.3, we can see that it decreases monotonically and smoothly, implying that good learning parameters have been found. The curve flattens out towards the end, which means that training the model for more epochs will not yield a significant increase in performance with the current set of hyperparameters.

Chapter 3

Problem C

3.1 Problem Description

Problem C consists in learning the solution operator that maps a thermal conductivity field $a(x, y)$ to a temperature field $u(x, y)$ in a heterogeneous solid material. The PDE governing this system is

$$-\nabla \cdot (a(x, y)\nabla u(x, y)) = f, \quad (x, y) \in \Omega = [0, 1]^2 \tag{3.1}$$

where $f = 10$ is a uniform heat source and Ω is the computational domain. The PDE must satisfy a constant temperature of 0 at the boundary $\partial\Omega$ of the computational domain, as expressed by the following boundary condition:

$$u(x, y) = 0, \quad (x, y) \in \partial\Omega \tag{3.2}$$

The provided dataset contains 1000 labeled pairs of conductivity fields $a(x, y)$ and temperature fields $u(x, y)$.

3.2 Methodology

3.2.1 The method selection

Since this problem involves learning a mapping from an arbitrary material field to a solution field, an operator is needed. Given that the dataset at hand is fully labeled, using a physics-informed DNO approach is not considered necessary in this case, as the network may learn the mapping completely through minimization of the data loss.

A visual analysis of the dataset for $a(x, y)$ shows that the material fields contain periodically-varying structures, as can be seen in Figure 3.1. This makes the Fourier Neural Operator (FNO) an excellent choice for this type of problem, as it can naturally learn the main frequency components in material and solution fields. Additionally, the FNO is computationally efficient, as it outputs a full solution field per forward pass, rather than a single point like DeepONet.

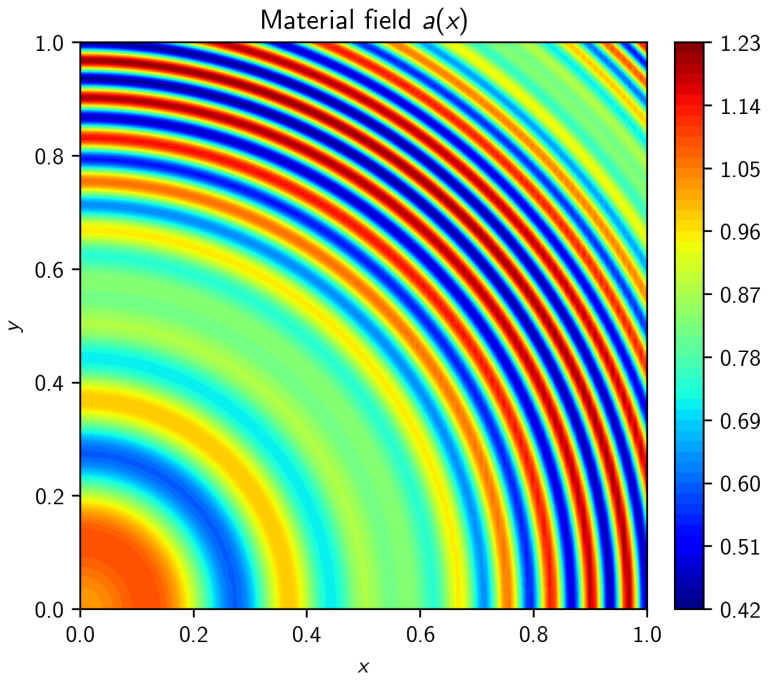


Figure 3.1: Example conductivity field $a(x, y)$

3.2.2 The loss function

The loss function for this problem is remarkably simple, given that no physics-enforcing loss terms are used. Thus, only a data loss is used to compare the prediction of the operator to the ground truth solution field. The total loss is given by

$$\mathcal{L}_\theta = \frac{1}{N} \sum_{i=1}^{N} \|\mathcal{G}_\theta\left(a_i(x, y)\right) - u_i(x, y)\|_2 \tag{3.3}$$

where $N = 1000$ is the number of training examples and $\mathcal{G}_\theta$ is the operator that maps the material field to the solution field: $\mathcal{G}_\theta : a(x, y) \mapsto u_\theta(x, y)$.

3.2.3 The network structure

The network structure used for Problem C is best described by Figure 3.2. As can be seen in the figure, the material field $a(x, y)$ is first "lifted" onto a higher dimensional space by a MLP with Linear activations, before being passed onto the Fourier layers, which perform the following operation:

$$v_{l+1} = \sigma_C \left[\mathcal{F}^{-1}\left(R_l \cdot \mathcal{F}(v_l)\right)(x) + W_l v_l(x)\right] \tag{3.4}$$

where v_l is the input to layer l, σ_C is an activation function, $\mathcal{F}$ denotes the Fourier transform, R_l is a tensor of learnable parameters and W_l is a tensor of learnable weights called the "bypass".

After the Fourier layers, the field is projected back to its final dimension by another MLP. This MLP consists of 3 layers, which temporarily change the channel dimension of the field from 40 to 128, then from 128 to 1, as can be seen in Figure 3.2. The activations for the 3 layers in this MLP are Linear, ReLu and Linear, respectively.

The specific parameters of each layer in the network can be found in Table 3.1.

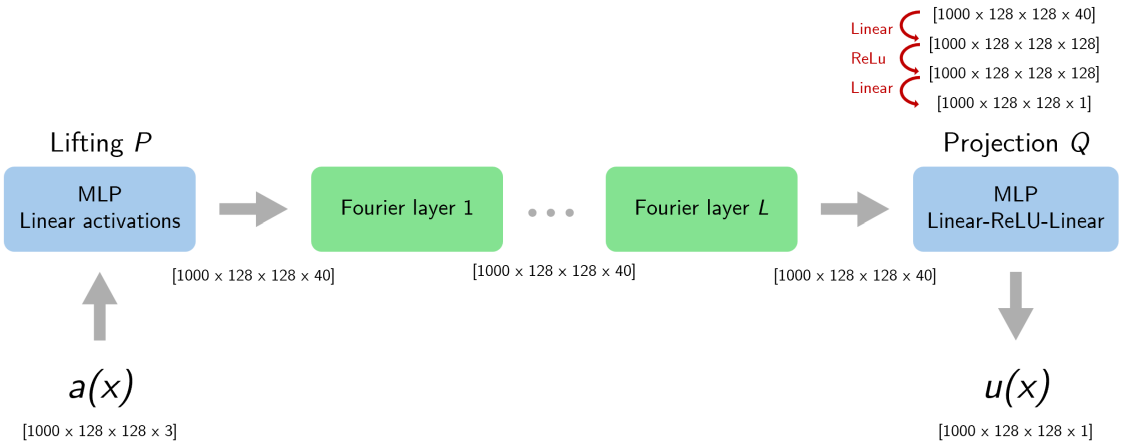


Figure 3.2: FNO structure for Problem C

Table 3.1: Network architecture of FNO for Problem C

Layer	Size / Neurons	Activation Function	Notes
Lifting Layer	40	Linear	-
Fourier Layer 1	40	$\sigma_C(x) = \text{ReLU}$	16 modes used
Fourier Layer 2	40	$\sigma_C(x) = \text{ReLU}$	16 modes used
Projection Layer 1	128	Linear	-
Projection Layer 2	128	ReLU	-
Projection Layer 2	1	Linear	-

3.2.4 The code link

Please provide the hyperlink (e.g., GitHub, Google Colab, or Kaggle Notebook) to your original code for solving this problem.

3.3 Numerical Results

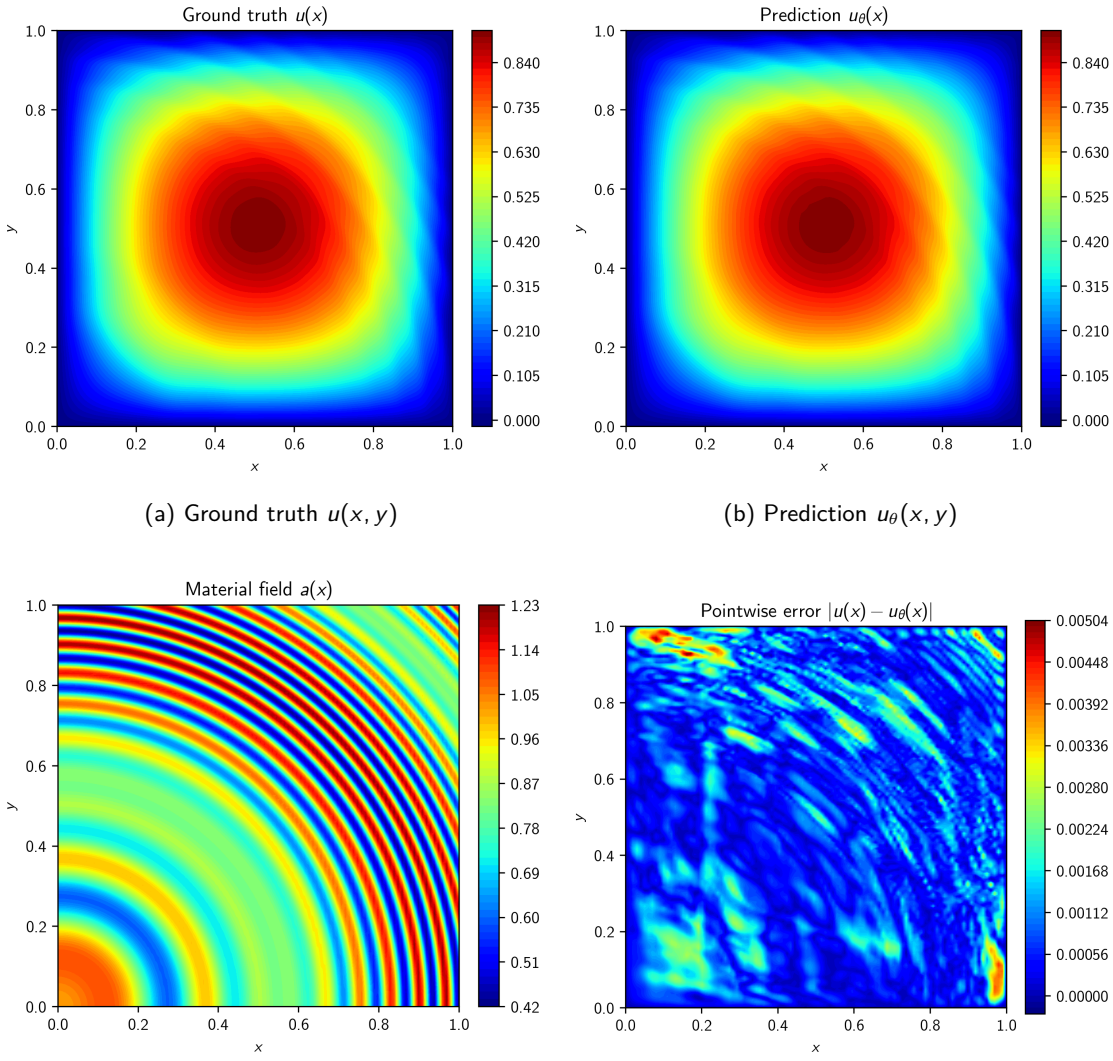
3.3.1 Experiment Setups

The hyperparameters used in the experimental setup are listed in Table 3.2.

Table 3.2: Numerical setup for training the FNO in PyTorch

Parameter	**Value**
Optimizer	Adam
Learning Rate (initial)	1×10^{-3}
Weight Decay	1×10^{-3}
Batch Size	5
Number of Epochs	30
Learning Rate Scheduler	StepLR
Decay Rate (α/N epochs)	0.5/7
Hardware	NVIDIA RTX 3050 Laptop (4GB)
Precision	Float32
Early Stopping	No

3.3.2 Numerical Results



(a) Ground truth $u(x, y)$

(b) Prediction $u_\theta(x, y)$

(c) Conductivity field $a(x, y)$

(d) Pointwise error $|u(x, y) - u_\theta(x, y)|$

Figure 3.3: Experiment results for the first test instance of Problem C

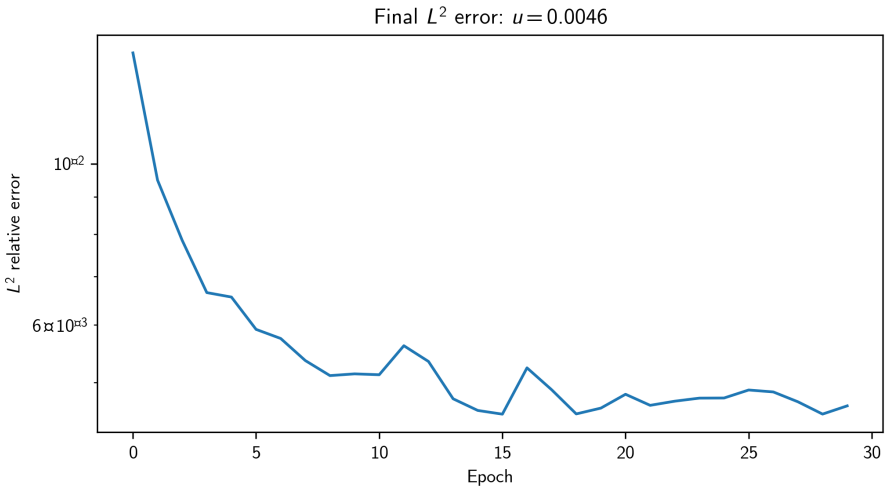


Figure 3.4: Evolution of L^2 relative error for Problem C

3.3.3 Discussion

As shown in Figure 3.3, the ground truth temperature field is approximated almost perfectly by the FNO. The absolute pointwise error achieved is mostly uniformly low, except for some regions where the conductivity field $a(x, y)$ has complex features. The conductivity field also has some patterns that are visually periodic/oscillatory, which the FNO excels at recognizing. This may be one of the reasons that the FNO works so well on this specific dataset.

Despite the training generally being slow (often reaching a few seconds per iteration), the L^2 relative error in Figure 3.4 appears to flatten out at just 20 iterations with the chosen hyperparameter set, meaning the convergence is rather fast. There do exist some oscillations in the curve, which implies that the model would benefit from more fine-tuning of the learning rate and its decay. This would likely lead to a lower final loss and thus a better performance.

Chapter 4

Problem D

4.1 Problem Description

Problem D consists in predicting the velocity field $u(x, t)$ of a road segment with traffic flow given an initial velocity profile $u(x, 0) = a(x)$. The governing PDE in this problem is Burgers' equation, namely

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \quad x \in (-1, 1), t \in (0, 1] \tag{4.1}$$

where $\nu = 0.1/\pi$ is the kinematic viscosity coefficient.

In addition to the previously mentioned initial condition $a(x)$, the PDE is subject to homogeneous Dirichlet boundary conditions at both ends of the road for the entire duration of time:

$$u(-1, t) = u(1, t) = 0, \quad t \in (0, 1] \tag{4.2}$$

The provided dataset is partially labeled, meaning that only some of the training examples $a(x)$ are paired with a corresponding ground truth solution field $u(x, t)$. Specifically, the dataset contains 200 labeled examples and 1800 unlabeled examples.

4.2 Methodology

4.2.1 The method selection

Similarly to Problem C, this problem involves learning an operator $\mathcal{G}_\theta$ from an input field to a solution field. In this case the dataset is not fully labeled, so a physics-informed DNO approach is necessary in order to extract information from the entire dataset.

For this purpose, the Physics-Informed Fourier Neural Operator (PINO) is selected. The reason for choosing PINO over PI-DeepONet is that the latter requires implementing 2 networks (branh net and trunk net), while the former requires just 1. Hence it was deemed simpler to implement, in addition to having less hyperparameters to be tuned.

4.2.2 The loss function

The loss function for Problem D consists of a data loss computed using the labeled examples and a PDE loss computed using all training examples. The reason a boundary loss is not used is that the Dirichlet conditions are directly enforced via a "boundary-vanishing mask" $m(x)$ that is later multiplied with the raw output of the network $\hat{u}_\theta(x, t)$:

$$m(x) = \cos\left(\frac{\pi x}{2}\right) \tag{4.3}$$

Since the mask is 0 at $x = \pm 1$, the predicted solution $u(x, t)$ will also be 0 at those spatial locations. Next, this product is multiplied by time t and added to the initial velocity profile $a(x)$ in order to also enforce that the initial condition is satisfied, as such:

$$u_\theta(x, t) = a(x) + t \cdot m(x)\hat{u}_\theta(x, t) \tag{4.4}$$

While this masking step is not strictly necessary for the network to function, it was found that the overall performance was improved, given that the network respects the boundary conditions from the very start of the training process.

The data loss is given by

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{lab}}} \sum_{i=1}^{N_{\text{lab}}} \|\mathcal{G}_\theta\left(a_i(x, y)\right) - u_i(x, y)\|_2 \tag{4.5}$$

where $N_{\text{lab}} = 200$ is the number of labeled training examples.

The PDE loss is computed as the mean of the sums of squared strong-form PDE residuals $\mathcal{R}\left[u_\theta; a\right]_{i,j}$ evaluated at all interior grid points for each sample in the batch. To approximate the derivatives in the residuals, central finite differencing is used, which is fast and consistent with PINO's grid-based representation:

$$\mathcal{R}\left[u_\theta; a\right]_{i,j} = \left.\frac{\partial u_\theta}{\partial t}\right|_{x_j, t_i} + u_\theta(x_j, t_i) \left.\frac{\partial u_\theta}{\partial x}\right|_{x_j, t_i} - \nu \left.\frac{\partial^2 u_\theta}{\partial x^2}\right|_{x_j, t_i} \tag{4.6}$$

$$\left.\frac{\partial u_\theta}{\partial t}\right|_{x_j, t_i} \approx \frac{u_\theta(x_j, t_{i+1}) - u_\theta(x_j, t_{i-1})}{2\Delta t} \tag{4.7}$$

$$\left.\frac{\partial u_\theta}{\partial x}\right|_{x_j, t_i} \approx \frac{u_\theta(x_{j+1}, t_i) - u_\theta(x_{j-1}, t_i)}{2\Delta x} \tag{4.8}$$

$$\left.\frac{\partial^2 u_\theta}{\partial x^2}\right|_{x_j, t_i} \approx \frac{u_\theta(x_{j+1}, t_i) - 2u_\theta(x_j, t_i) + u_\theta(x_{j-1}, t_i)}{2\Delta x^2} \tag{4.9}$$

The PDE loss is then given by

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{train}} N_x N_t} \sum_{i=1}^{N_{\text{train}}} \sum_{j,k} \left|\mathcal{R}\left[\mathcal{G}_\theta(a_i); a_i\right]_{j,k}\right|^2 \tag{4.10}$$

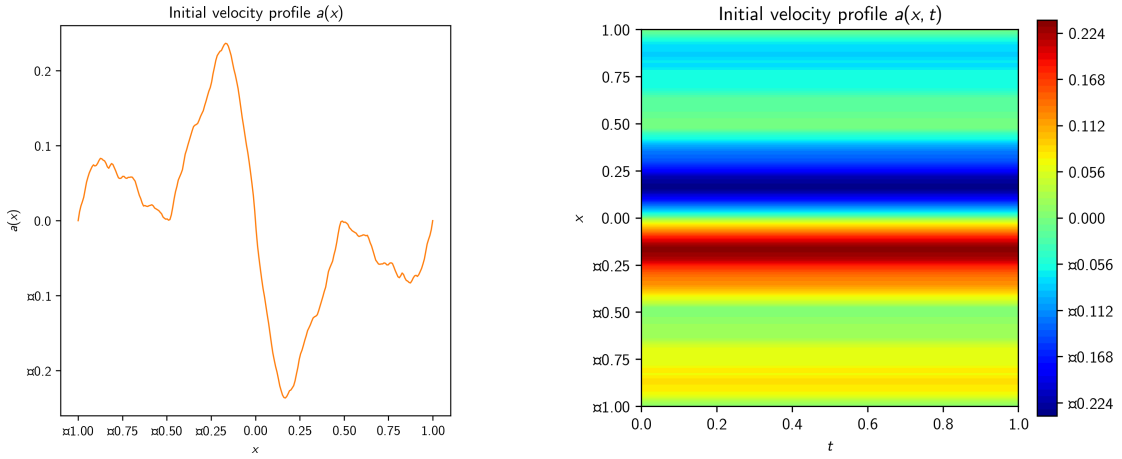
where $N_{\text{train}} = N_{\text{lab}} + N_{\text{unlab}} = 200 + 1800 = 2000$ is the total number of training examples (labeled + unlabeled), $N_x = 256$ is the spatial resolution of the velocity field and $N_t = 200$ is the temporal resolution of the velocity field.

Finally, the total training loss $\mathcal{L}_\theta$ is the weighted sum of both losses, where the weights associated to each loss are hyperparameters.

$$\mathcal{L}_\theta = w_{\text{data}}\mathcal{L}_{\text{data}} + w_{\text{PDE}}\mathcal{L}_{\text{PDE}} \tag{4.11}$$

4.2.3 The network structure

In order to slightly facilitate the implementation of this PINO, the data for the initial velocity profiles $a(x)$ was extruded across the time dimension, resulting in 2-dimensional fields $a(x, t)$, as can be seen in Figure 4.1. The reasoning for this choice was that this extrusion could provide information about the initial condition of the PDE to the network at every point in time, in addition to not requiring a change of shape during the lifting or projection steps. This way, the mapping is 2D to 2D.



(a) 1D velocity profile $a(x)$

(b) Extruded 2D velocity profile $a(x, t)$

Figure 4.1: Extrusion of $a(x)$ to the time dimension

The network structure used for Problem D is conceptually identical to the one used in Problem C and described in Figure 3.2. Only some minor changes have been made to the size of the model, which are summarized in Table 4.1.

Table 4.1: Network architecture of PINO for Problem D

Layer	Size / Neurons	Activation Function	Notes
Lifting Layer	20	Linear	-
Fourier Layer 1	20	$\sigma_D(x) = \text{ReLU}$	8 modes used
Fourier Layer 2	20	$\sigma_D(x) = \text{ReLU}$	8 modes used
Projection Layer 1	128	Linear	-
Projection Layer 2	128	ReLU	-
Projection Layer 2	1	Linear	-

4.2.4 The code link

Please provide the hyperlink (e.g., GitHub, Google Colab, or Kaggle Notebook) to your original code for solving this problem.

4.3 Numerical Results

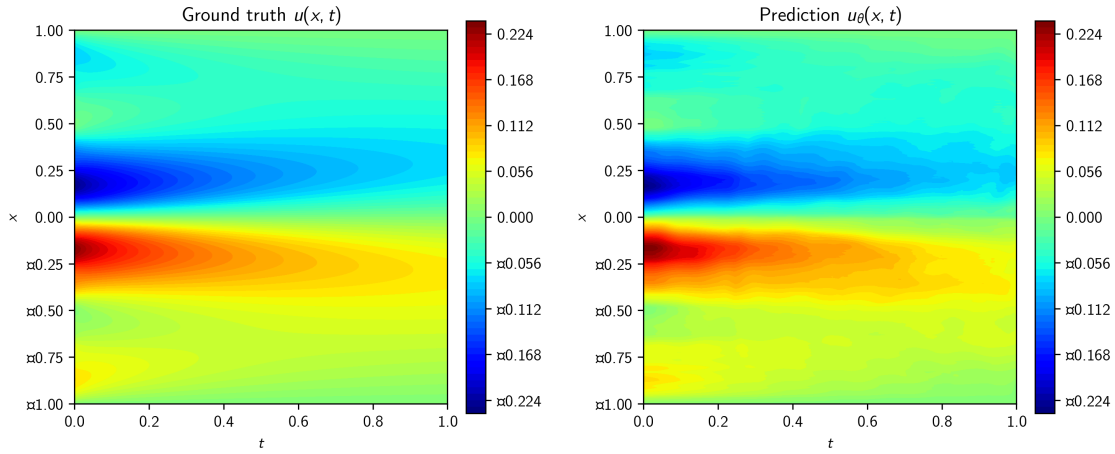
4.3.1 Experiment Setups

The hyperparameters used in the experimental setup are listed in Table 4.2.

Table 4.2: Numerical setup for training the PINO in PyTorch

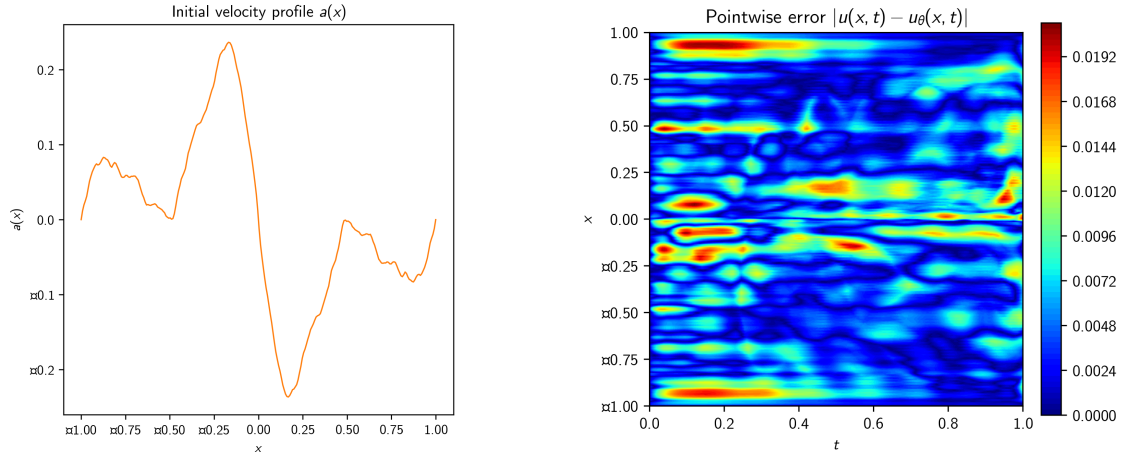
Parameter	**Value**
Optimizer	Adam
Learning Rate (initial)	5×10^{-4}
Weight Decay	1×10^{-4}
Batch Size	10
Number of Epochs	30
Learning Rate Scheduler	StepLR
Decay Rate (α/N epochs)	0.9/10
Hardware	NVIDIA RTX 3050 Laptop (4GB)
Precision	Float32
Early Stopping	No

4.3.2 Numerical Results



(a) Ground truth $u(x, t)$

(b) Prediction $u_\theta(x, t)$



(c) Initial velocity profile $a(x)$

(d) Pointwise error $|u(x, t) - u_\theta(x, t)|$

Figure 4.2: Experiment results for the first test instance of Problem D

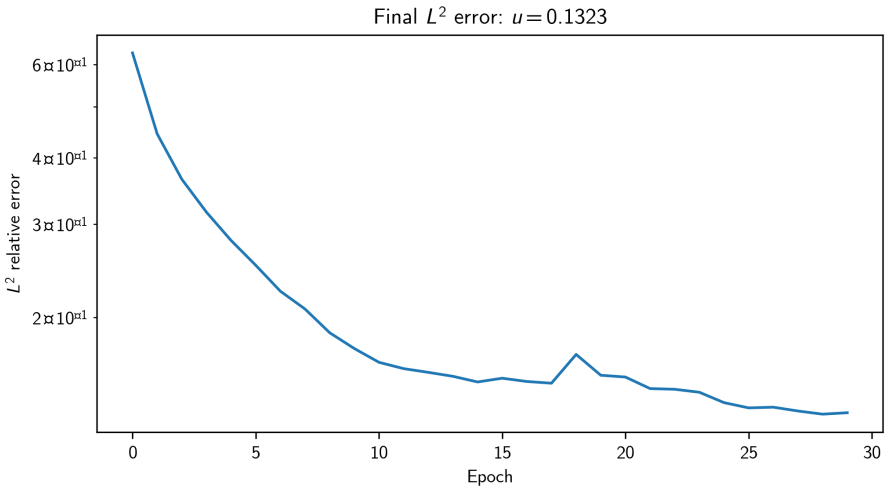


Figure 4.3: Evolution of L^2 relative error for Problem D

4.3.3 Discussion

As can be observed in Figure 4.2, the PINO does a good job at approximating the velocity field $u(x, t)$, primarily in terms of its overall shape and structure. The diffusive behavior is captured well, with the peaks and valleys in the spatial velocity profile smoothing out over time. As expected, the boundary conditions are perfectly respected, since this was forced to happen due to the application of the mask function in equation (4.4). However, the smoothness of the ground truth solution has not been completely captured, as the pointwise error field is somewhat noisy.

Similarly to the regular FNO used for Problem C, the training process for the PINO was very slow, but converged in as little as 30 epochs, as we can observe in Figure 4.3. This time the convergence is smoother, hinting at well-tuned learning rate parameters. With the current number of epochs the L^2 relative error is just starting to flatten out, meaning the model may get some increase in performance by training it for longer, though this increase may not be significant.

Chapter 5

Conclusions

5.1 Summary of applied methods

For Problem A, the VPINN approach proved very well-suited as it incorporated the weak form. This avoided having to compute any higher order derivatives, and even the 1st order ones were known since they come from Legendre polynomials. This gives VPINN a strong advantage over a regular PINN in this type of problem.

For Problem B, ParticleWNN initially stood out to me as the method of choice to approximate the heterogeneous pressure field. My hope was that the locally-supported test functions could approximate the randomly distributed features of the material field without affecting the rest of the domain. However, even this type of network struggled to complete the task to its full extent and required tedious fine-tuning of hyperparameters to get acceptable performance. Perhaps a different type of physics-informed method could have achieved better performance.

In Problem C, the FNO approach yielded excellent results with very few training epochs. The network seemed to be very successful at capturing periodic spatial patterns in a way that a point-wise MLP would likely not be able to do. Furthermore, the fact that this dataset is fully labeled and does not call for a PDE loss term establishes FNO as the undisputed method of choice for this problem.

Finally, the PINO method showed very decent performance on Problem D, albeit with the help of the boundary-vanishing mask that hard-enforced boundary conditions and initial conditions. It was also the slowest of all 4 methods during the training phase, but much like the regular FNO it converged quickly. Despite not achieving an extreme level of accuracy, the PINO still remains a very practical option for engineering tasks due to its ability to work with semi-supervised datasets.

5.2 Reflections

After completing this project, it is clear to me that each PDE-related problem requires careful analysis in order to determine which methods are applicable, and even further analysis to choose which method is the most suitable out of the available ones. Nevertheless, this decision is not always obvious, and it is often the case that multiple methods must be tried before one is found that gives acceptable performance. Because of this, I found the complexity of implementation to be one of the most decisive factors when choosing a deep learning method.

Alongside the implementation complexity is the training time requirement, which unfortunately can only be gauged once a method has already been fully implemented. Although the training time may be somewhat predictable by theory, it is impossible to know exactly

how well a model will perform on a dataset in advance, which may force you to increase the number of model parameters. This will lead to increases in training time, making this time highly unpredictable. The dimensionality of the data should certainly be considered here.

Another theme I found to be recurring across all problems is the trade-off between the expressive power of a network (which relates to parameter count) and its train-ability. Larger models can capture more complex behavior at the cost of slower iterations, which may be an issue depending on the hardware one has available. During training, I often ran into CUDA memory limitations or situations where the transfer of data to the GPU was simply too slow to be practical, which forced me into reducing batch sizes and other downgrades.

Overall, I found the application of deep learning methods to PDEs to be a very interesting innovation. I think they have a very significant potential to compete with classical methods in scenarios where many repeated computations of the same PDE family are needed. I also believe these methods will only continue to get better in the future as machine learning is a field with extremely fast development right now, while classical methods have been around for much longer and will likely not see much more innovation. On the other hand, the lack of a convergence guarantee and their implementation complexity are major drawbacks of deep learning methods in my opinion. They are also more inaccessible to those engineers, researchers and scientists without a machine learning background, while classical methods are currently more widespread.

5.2.1 Potential improvements

Some of the potential improvements that could be made to my implementations are the following:

- Adapting the loss weighting during training to change the relative importance of each loss term at each stage of the training process. This would allow, for instance, initially learning to satisfy the boundary conditions without worrying about internal points, then increasing the PDE loss to satisfy the equation across the whole domain.
- Performing data augmentation by rotating, reflecting or cropping the training and testing data in order to have more diverse datasets. This would make the models generalize better.
- Training multiple copies of the same model with different weight initializations and averaging their predictions to reduce variance.

References

[1] V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell, and G. Wetzstein, *Implicit Neural Representations with Periodic Activation Functions*, arXiv:2006.09661, 2020